



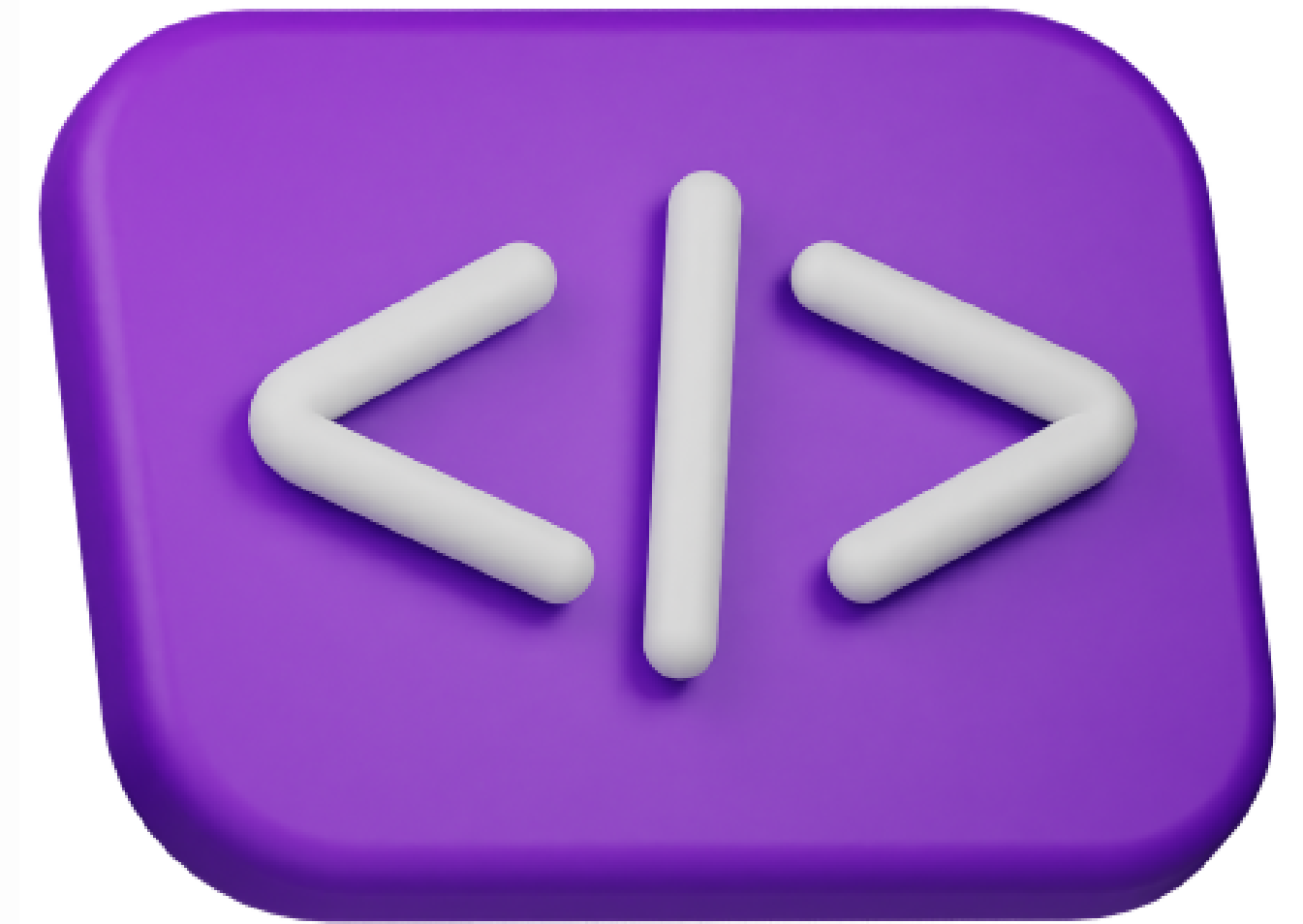
Sass

Lecture 2



TABLE OF CONTENT

- ▶ What is Sass
- ▶ Benefits of Using Sass
- ▶ Starting Sass
- ▶ Features of Sass



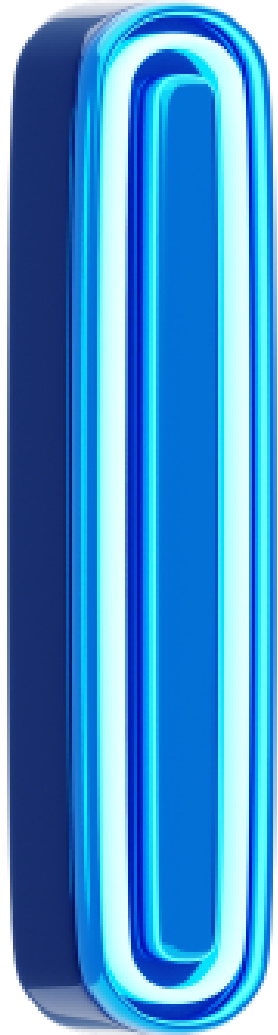
What is Sass?

Sass (Syntactically Awesome Stylesheets) as a **CSS** preprocessor that extends **CSS** capabilities with features like *variables*, *nesting*, and *mixins*.

created by Hampton Catlin in 2006



Benefits of Using Sass



Enhanced maintainability :

Makes large projects easier to manage.

When working on large projects, managing CSS files can become challenging. Styles can grow unwieldy, leading to difficulties in updating or debugging. Sass introduces features like variables, nesting, and modularization, which help structure styles logically

Benefits of Using Sass



Reusability :

Variables and mixins prevent repetitive code. Sass allows developers to reuse code through variables and mixins, reducing repetition and making the codebase cleaner and more efficient

Variables

store values like colors, fonts, or dimensions that can be reused across stylesheets.

Mixins

let you define reusable chunks of CSS that can be included in multiple selectors

Benefits of Using Sass



Cleaner code :

Nesting helps organize styles hierarchically.

Sass allows you to nest your styles, which mirrors the structure of your HTML. This hierarchical organization makes the code more readable and intuitive, especially for complex designs with deeply nested elements.

Benefits of Using Sass



Faster development :

Partials and imports allow breaking stylesheets into smaller files.

Sass supports partials and imports, allowing you to split your styles into smaller, modular files. Each partial contains styles for a specific part of the project (e.g., `_header.scss` for the header, `_footer.scss` for the footer). These partials are then combined into a single CSS file using `@import`.

Starting Sass

1. Install these extensions

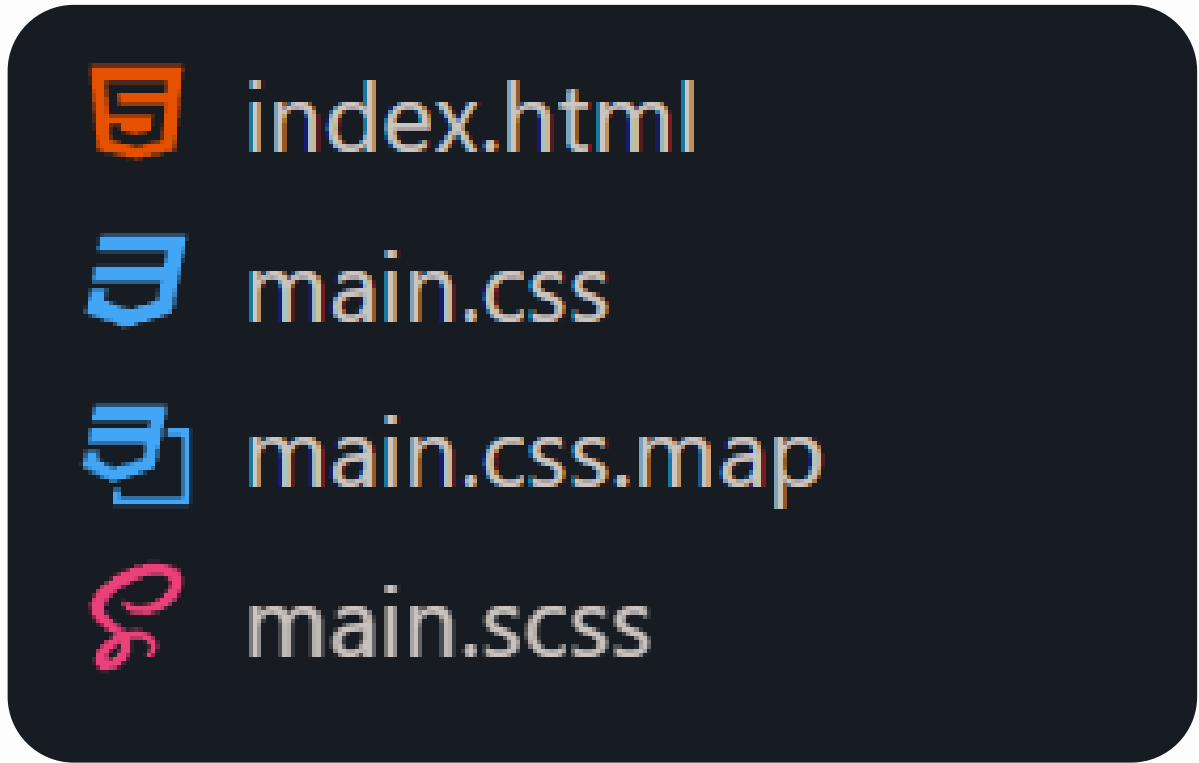


Live Sass Compiler



Sass (.sass only)

2. Create a scss file and watch a sass compiler

A dark-themed snippet representing a VS Code file explorer. It lists four files with their respective icons: index.html (HTML icon), main.css (CSS icon), main.css.map (CSS map icon), and main.scss (Sass icon).

```
index.html  
main.css  
main.css.map  
main.scss
```

After watching sass 2 file will automatically creates

1. CSS file for connecting CSS in HTML
2. CSS .map file for translating scss fro css

What is a variable in Sass?

A variable in Sass is a way to store values (like colors, fonts, or sizes) so you can reuse them throughout your stylesheet. This makes your code easier to manage and update.

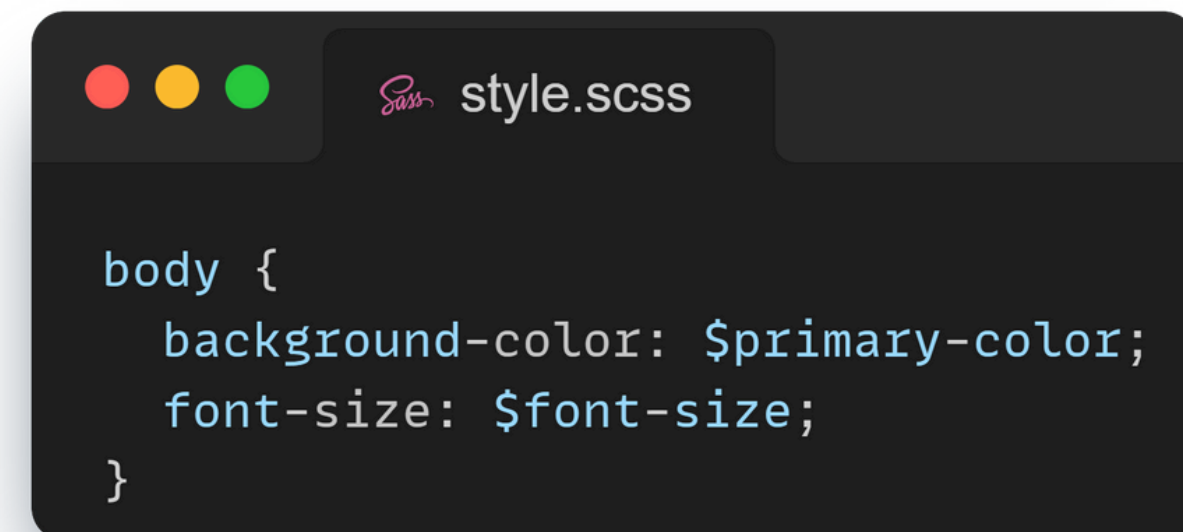
How to define a variable?

A code editor window with a dark theme. The title bar shows three colored circles (red, yellow, green) and a tab labeled 'style.scss' with a Sass logo. The code content is:

```
$primary-color: #3498db;  
$font-size: 16px;
```

```
$primary-color: #3498db;  
$font-size: 16px;
```

How to use a variable?

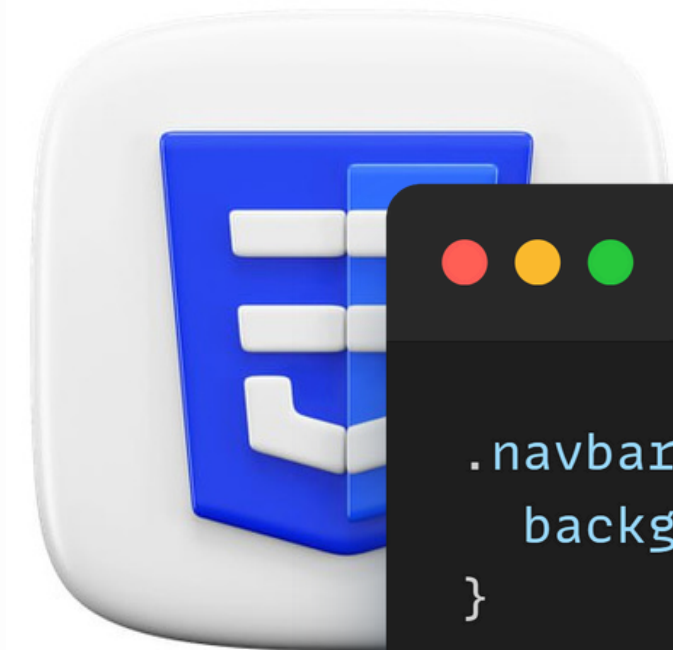
A code editor window with a dark theme. The title bar shows three colored circles (red, yellow, green) and a tab labeled 'style.scss' with a Sass logo. The code content is:

```
body {  
  background-color: $primary-color;  
  font-size: $font-size;  
}
```

```
body {  
  background-color: $primary-color;  
  font-size: $font-size;  
}
```

Nested Rules

Nested rules in **Sass** allow you to write CSS in a hierarchical structure that follows the HTML structure. This makes the code more readable and easier to maintain.



```
style.css

.navbar {
  background-color: #333;
}

.navbar ul {
  list-style: none;
}

.navbar ul li {
  display: inline-block;
}
```



```
style.scss

.navbar {
  background-color: #333;

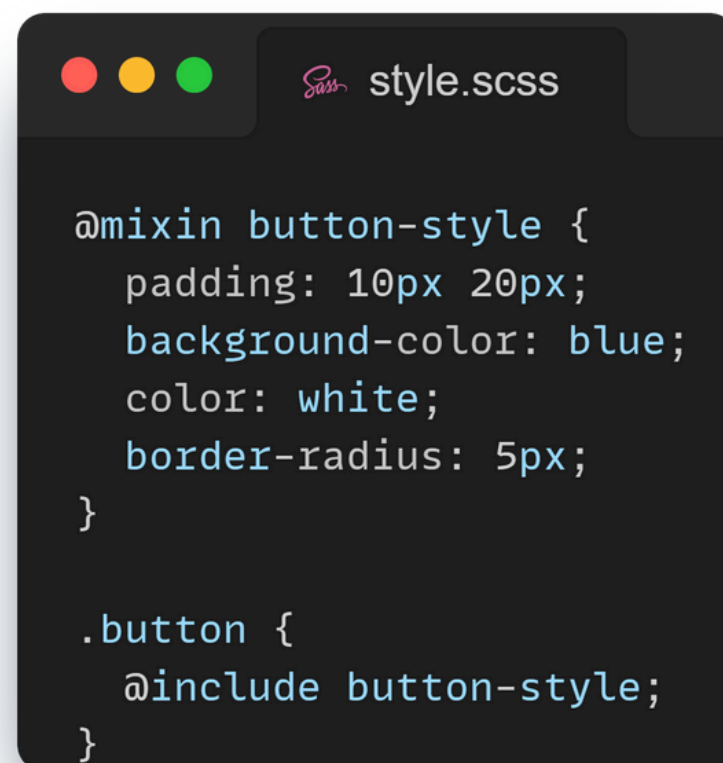
  ul {
    list-style: none;

    li {
      display: inline-block;
    }
  }
}
```

What is a **@mixin** in Sass?

A **mixin** is a reusable block of CSS code that you can use anywhere in your styles. It helps avoid repeating code and makes your styles more flexible.

Without params



```
style.scss

@mixin button-style {
  padding: 10px 20px;
  background-color: blue;
  color: white;
  border-radius: 5px;
}

.button {
  @include button-style;
}
```

With params



```
style.scss

@mixin custom-button($color, $size) {
  background-color: $color;
  font-size: $size;
  padding: 10px 15px;
}

.primary {
  @include custom-button(blue, 16px);
}

.secondary {
  @include custom-button(green, 14px);
}
```

Use **@mixin** to define a mixin and **@include** to use it.

Mixins can take values (parameters) to make them more flexible

You can use **@if** inside **mixins** to apply styles only when a condition is met. This makes your styles more dynamic and flexible.

Mixin

```
@mixin button-style($type) {  
  padding: 10px 20px;  
  border-radius: 5px;  
  color: white;  
  
  @if $type = "primary" {  
    background-color: blue;  
  } @else if $type = "secondary" {  
    background-color: green;  
  } @else {  
    background-color: gray;  
  }  
}
```

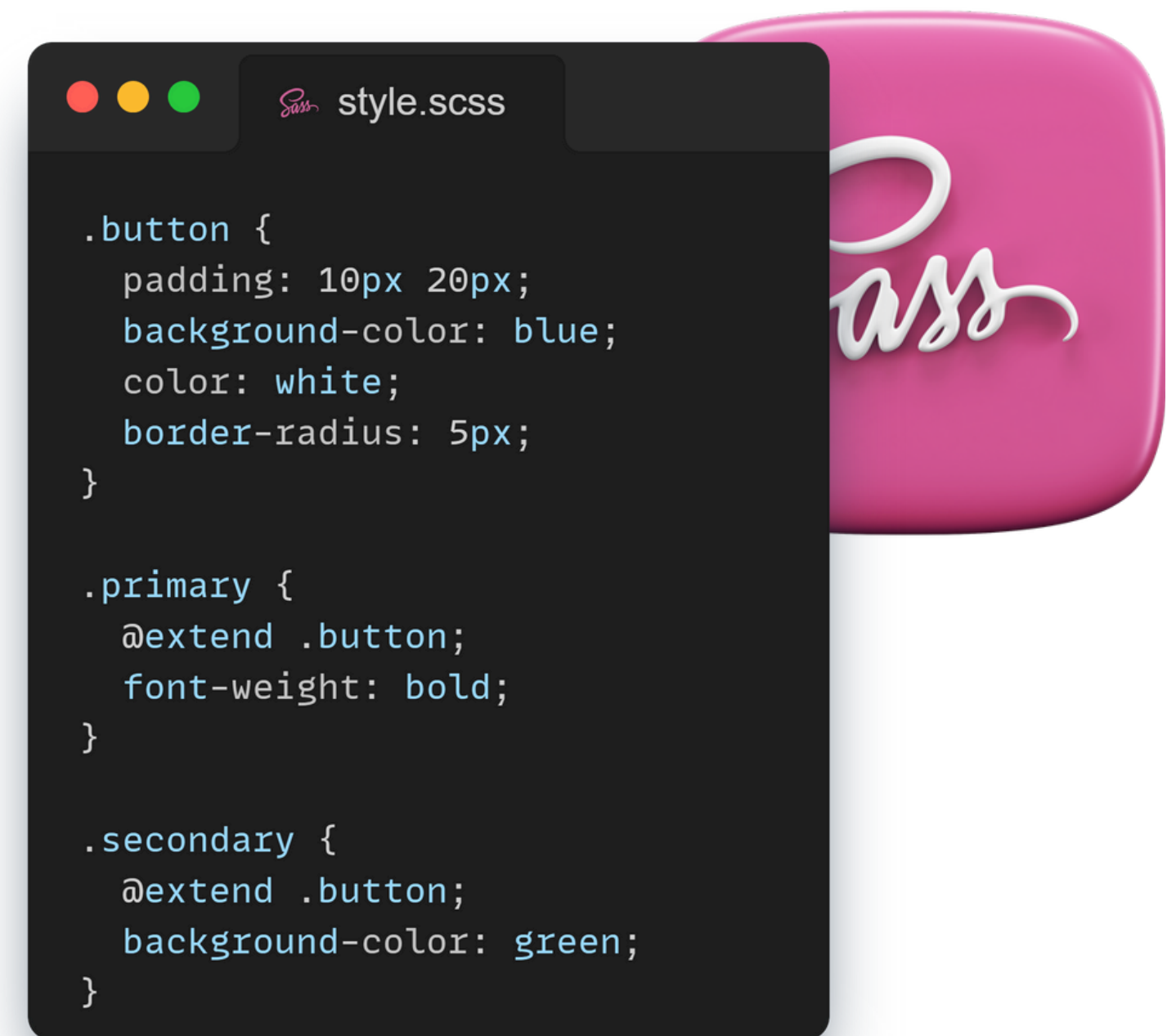
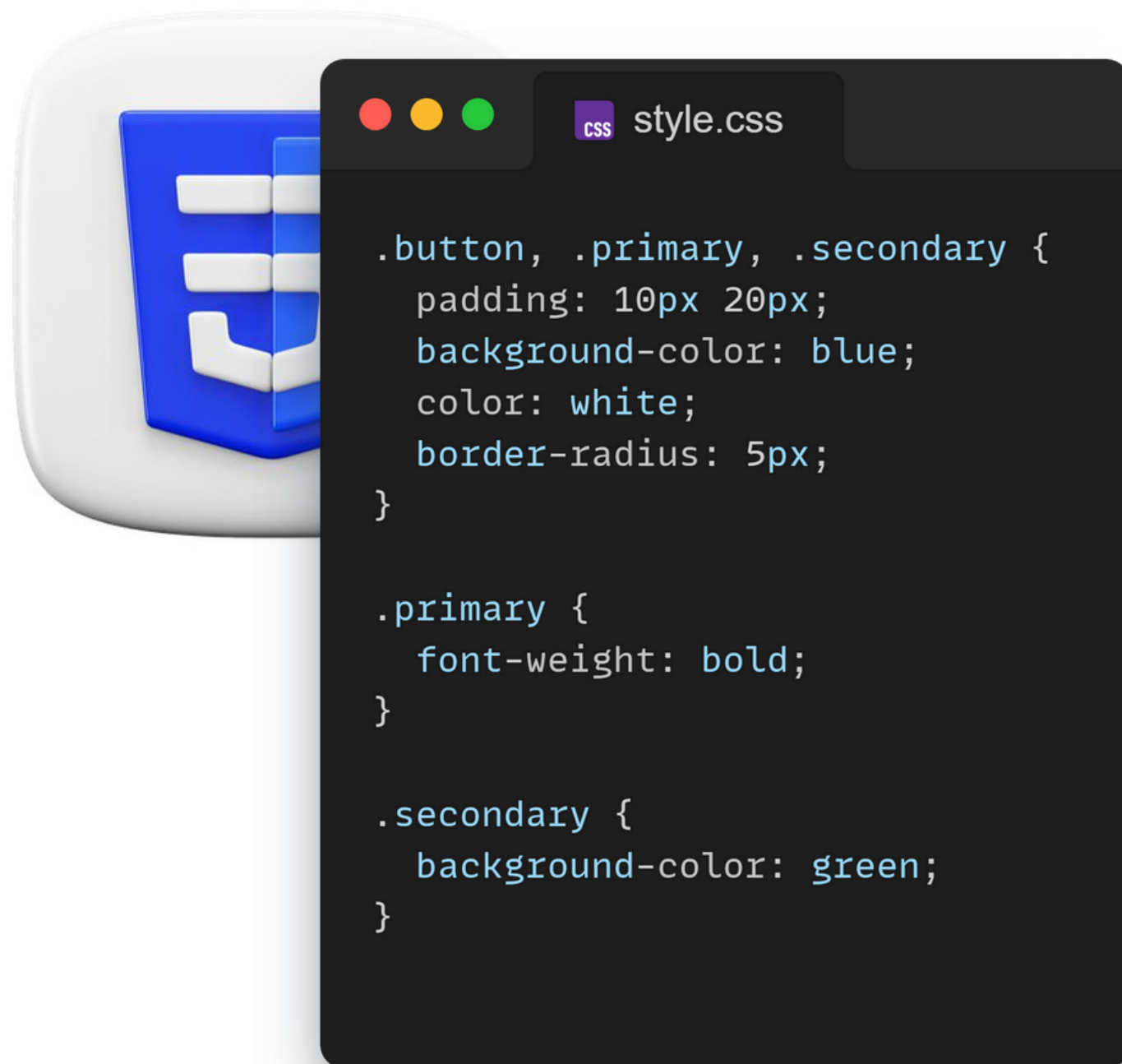
Usage

```
.primary {  
  @include button-style("primary");  
}
```

```
.secondary {  
  @include button-style("secondary");  
}
```

What is **@extend** in Sass?

The **@extend** feature allows one selector to inherit the styles of another. This helps reduce duplicated code and keeps styles more organized.



THANK YOU!

Diamonds are born under pressure.
If you can handle this, you will
become a diamond.

